

ASSIGNMENT 2_2

M.parimala

2303A510H5

Batch:30

Task 1:

Code & output:

```
27 27 ass2.2.py / ...
#Use Gemini in Colab to generate a function that filters out all negative numbers from a list.
def filter_negative_numbers(numbers):
    return [num for num in numbers if num >= 0]
# Example usage
my_list = [1, -2, 3, -4, 0, 5, -6]
filtered_list = filter_negative_numbers(my_list)
print(f"original list: {my_list}")
print(f"Filtered list(non-negative numbers): {filtered_list}")

18.0-WIN32-x64\bundled\libs\debugpy\launcher 63683 -- C:\Us
original list: [1, -2, 3, -4, 0, 5, -6]
Filtered list(non-negative numbers): [1, 3, 0, 5]
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

Filtering out negative values ensures only valid sensor readings are processed, improving data reliability. This task highlights how AI can quickly generate simple yet useful functions for IoT applications. It demonstrates the importance of preprocessing before analysis.

Task 2:

Code & output:

```

#generate a Python function that counts vowels, consonants, and digits in a string
def count_vowels_consonants_digits(input_string):
    vowels = "aeiouAEIOU"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0
    for char in input_string:
        if char.isalpha(): # Check if the character is an alphabet
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
        elif char.isdigit(): # Check if the character is a digit
            digit_count += 1
    return vowel_count, consonant_count, digit_count

# Example usage
input_string = "Hello World! 123"
vowels, consonants, digits = count_vowels_consonants_digits(input_string)
print(f"Input string: {input_string}")
print(f"Number of vowels: {vowels}")

```

```

10.0-WIN32-x64 (bundled) (lib3 (debugpy (launcher) 35250)
Input string: Hello World! 123
Number of vowels: 3
PS C:\Users\parim\Ai Assistant coding>

```

Justification:

Counting vowels, consonants, and digits builds a foundation for text analytics. AI-generated code helps automate repetitive logic and ensures accuracy. This task shows how Copilot/Gemini can simplify character classification while maintaining readability.

Task 3:

Code & output:

```
import re

def is_palindrome(s):
    # Remove all non-alphanumeric characters and convert to lowercase
    cleaned_string = re.sub(r'^a-zA-Z0-9', '', s).lower()
    return cleaned_string == cleaned_string[::-1]

# Example usage
test_string = "A man, a plan, a canal: Panama"

if is_palindrome(test_string):
    print(f'"{test_string}" is a palindrome.')
else:
    print(f'"{test_string}" is not a palindrome.')
```

```
18.0-WIN32-x64(bundled\libs\debugpy\launcher 5856) -- C:\Use
"A man, a plan, a canal: Panama" is a palindrome.
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

Comparing outputs from Gemini and Copilot reveals differences in clarity, structure, and coding style. This task emphasizes how multiple AI tools can solve the same problem but with varying readability. It helps developers choose the most suitable tool for their workflow.

Task 4:

Code & output:

```
def is_palindrome_method1(input_string):
    """
    Checks if a string is a palindrome by reversing the string and comparing it to the original.

    Args:
        input_string (str): The string to check.

    Returns:
        bool: True if the string is a palindrome, False otherwise.
    """
    reversed_string = input_string[::-1]
    return input_string == reversed_string
```

Line-by-line explanation:

`def is_palindrome_method1(input_string):`

This line defines a Python function named `is_palindrome_method1` that takes one argument: `input_string` (which is expected to be a string). `"""..."""`

This is a docstring, which provides a brief description of what the function does, its arguments (Args), and what it returns (Returns). It's good practice for documenting code. `reversed_string = input_string[::-1]`

This is the core of the function. It creates a new string called `reversed_string`. `input_string[::-1]` uses Python's string slicing feature: `[start:end:step]`. Omitting start and end means to take the whole string. A step of `-1` means to iterate through the string in reverse order. So, this line effectively reverses the `input_string` and assigns it to `reversed_string`. `return input_string == reversed_string`

This line compares the original `input_string` with its `reversed_string`. The `==` operator checks for equality. If the original string is identical to its reversed version (meaning it's a palindrome), the comparison evaluates to `True`. Otherwise, if they are different, it evaluates to `False`. The function then returns this `True` or `False` boolean value.

Justification:

AI explanations make unfamiliar code easier to understand by breaking it down line by line. This improves learning and debugging efficiency. The task demonstrates how AI can act as a teaching assistant, bridging gaps in comprehension for students and developers.